

DOCUMENTO DE DISEÑO DE SOFTWARE (SDD)

Proyecto: Plataforma Integral de Gestión y Reporteo Trimestral (REDECO/CONDUSEF)
Institución: Unión de Crédito Integral, S.A. de C.V. (UCISA)
Curso/Asignatura: Negocios Electrónicos.
Fecha de Publicación: 13 de junio del 2026
Versión: 1.0.0

CONTROL DE VERSIONES Y AUTORES

Autores / Integrantes del Proyecto:

1. Adán Gurrola Grijalva - 38198801
2. Eduardo Octavio Picazo Méndez - 42208075
3. Fredy Rodriguez Rodarte - 42203151

Historial de Revisiones:

Versión	Fecha	Descripción del Cambio	Autor(es)	Revisor / Aprobador
1.0.0	13-jun.-26	Estructuración del diseño detallado del sistema, arquitectura física, lógica, modelos relacionales y diagramas de flujo.	Adán Gurrola. Eduardo Octavio. Fredy Rodriguez	Carlos Eduardo Acuña López

1. INTRODUCCIÓN

1.1 Propósito y Ámbito del Diseño

Este Documento de Diseño de Software (SDD, por sus siglas en inglés, *Software Design Document*) describe a nivel técnico, lógico y estructural la arquitectura del sistema REDECO-UCISA. Su propósito es servir de plano guía para el desarrollo, auditoría y posterior mantenimiento del sistema, detallando cómo los requisitos establecidos en la especificación de requisitos (SRS) son traducidos en componentes de software específicos.

El ámbito de este diseño abarca la arquitectura de la base de datos relacional de la plataforma, el flujo lógico de sus controladores, el diseño detallado de la interfaz gráfica adaptada a impresión de comprobantes, y el empaquetado final de datos para la API de CONDUSEF.

1.2 Resumen de la Arquitectura Seleccionada

La plataforma de software REDECO-UCISA está estructurada bajo el patrón arquitectónico **Modelo-Vista-Template (MVT)**, una variante altamente acoplada al patrón **Modelo-Vista-Controlador (MVC)** nativa del framework web Django.

- **Modelo (Model):** Representa la abstracción de las tablas relacionales de la base de datos mediante clases de Python, gestionando los tipos de datos y la integridad referencial.
- **Vista (View):** Actúa como el controlador de la lógica de negocio. Recibe solicitudes HTTP, realiza consultas e interacciones con los modelos y determina qué información enviar de vuelta.
- **Plantilla (Template):** Capa de presentación renderizada dinámicamente en el servidor (Server-Side Rendering) y potenciada con interactividad AJAX local en el cliente.

El sistema se define como un **Monolito Modular**, ya que todo el código del backend, frontend, archivos estáticos y configuración de base de datos se ejecutan en un mismo proceso del servidor web, pero separados lógicamente por "aplicaciones" independientes (`consultas` y `socios`).

2. DISEÑO DEL SISTEMA Y ARQUITECTURA GLOBAL

2.1 Vista Arquitectónica

El sistema opera en una infraestructura clásica de tres capas lógicas (Presentación, Lógica de Negocio y Persistencia de Datos). A continuación, se detalla el comportamiento de cada una de estas capas y su interacción mutua:

1. Capa de Presentación (Frontend):

Construida con plantillas HTML5 extendidas de un contenedor común (`base.html`). Implementa la biblioteca de estilos Bootstrap 5.3 para renderizar componentes visuales adaptables (responsive) y soportar de forma nativa el modo oscuro mediante atributos `data-bs-theme`. Los scripts de cliente escritos en Vanilla JS realizan peticiones asíncronas (`fetch`) en segundo plano para obtener datos geográficos sin recargar la página y gestionan la lógica de confirmación mediante modales dinámicos.

2. Capa de Lógica de Negocio y Control (Backend):

Procesada por el intérprete de Python 3.12 y el servidor de aplicaciones WSGI/ASGI de Django. Esta capa gestiona las rutas a través del mapeador de URLs del proyecto, sanitiza y valida los datos de entrada por medio de objetos `ModelForm` y ejecuta las reglas de negocio críticas, como la asignación del folio único mensual y el empaquetado asíncrono en lotes JSON.

3. Capa de Persistencia (Base de Datos):

Utiliza el motor SQL relacional ligero SQLite embebido en el sistema de archivos del servidor. A través de la capa de abstracción del ORM de Django, se realizan consultas SQL transaccionales complejas, optimizadas para el catálogo geográfico SEPOMEX mediante índices a nivel físico.

4. Conexión Externa (API CONDUSEF):

La integración se realiza de forma desacoplada y asíncrona. El sistema exporta la información en un formato JSON estándar que cumple con la estructura requerida por los tokens de acceso provistos para el sector "UNIONES DE CREDITO" de la CONDUSEF (institución clave 1286).

2.2 Descomposición en Subsistemas/Módulos

El proyecto REDECO se divide estructuralmente en los siguientes directorios principales a nivel de código fuente:

- `NE_REDECO/NE_REDECO/`: Directorio raíz de configuración general del proyecto.
 - `settings.py`: Contiene las variables globales del sistema, configuración del motor de base de datos (`sqlite3`), middleware de seguridad (CSRF, protección Clickjacking), idioma y zona horaria.
 - `urls.py`: Declaración del enrutador central (URL dispatcher) que mapea los paths del navegador web hacia los controladores correspondientes.
- `NE_REDECO/apps/socios/`: Subsistema encargado de la gestión de entidades financieras afiliadas y catálogos de códigos postales.
 - `models.py`: Modelado físico de la entidad `Socio` y del catálogo geográfico `CatalogoSepomex`.
 - `views.py`: Lógica para procesar la lista de socios y el endpoint API REST local `/buscar-cp/` de SEPOMEX.
 - `urls.py`: Declaración de rutas internas del módulo.
- `NE_REDECO/apps/consultas/`: Subsistema núcleo (core) que gestiona el flujo de quejas y la exportación regulatoria.
 - `models.py`: Definición de las entidades `Producto`, `Causa` y la entidad transaccional `Queja`.
 - `forms.py`: Clase `QuejaForm` que define los metadatos y widgets visuales de validación de quejas.

- `views.py`: Controladores para registrar quejas (`captura_queja`), renderizar tickets de acuse (`detalle_ticket`) y empaquetar JSON de cierres (`cierre_trimestral`).
 - `NE_REDECO/templates/`: Capa de presentación y recursos estáticos. Contiene subcarpetas para las plantillas específicas de cada aplicación y el archivo `base.html` que define la cabecera, navegación común, pie de página y la lógica de conmutación de modo oscuro persistida en el navegador.
-

3. DISEÑO DETALLADO DE COMPONENTES

3.1 Modelos de Datos / Entidades

A continuación, se detalla el esquema relacional derivado de las clases del modelo del ORM de Django:

3.1.1 Entidad `Socio` (Tabla `socio`)

Representa a las instituciones asociadas o sucursales sobre las cuales recae la responsabilidad de la queja.

- `id`: `BigAutoField` (Entero autoincremental de 64 bits, Clave Primaria por defecto).
- `clave_condusef`: `CharField(max_length=50)` (Cadena de caracteres, Clave Única a nivel de base de datos).
- `nombre_institucion`: `CharField(max_length=255)`.
- `tipo_entidad`: `CharField(max_length=50)` (Elecciones restringidas: 'Caja de Ahorro', 'Sucursal', 'Sofipo', 'Banco', 'Otro').
- `codigo_postal`: `CharField(max_length=5)` (Nullable, código postal de ubicación del socio).
- `estado_republica`: `CharField(max_length=50)` (Elecciones restringidas a los 32 estados de la República Mexicana).
- `municipio`: `CharField(max_length=150)` (Nullable).
- `colonia`: `CharField(max_length=150)` (Nullable).
- `estatus`: `CharField(max_length=15)` (Elecciones restringidas: 'Activo', 'Inactivo', por defecto 'Activo').
- `fecha_registro`: `DateTimeField` (Generado automáticamente al insertar el registro, inmutable).

3.1.2 Entidad `CatalogoSepomex` (Tabla `catalogo_sepomex`)

Contiene los registros del catálogo nacional postal para la validación geográfica.

- `id`: `BigAutoField` (Clave Primaria).
- `codigo_postal`: `CharField(max_length=5)` (Indexado físicamente mediante índice de base de datos denominado `idx_codigo_postal`).
- `clave_entidad_federativa`: `IntegerField`.
- `descripcion_entidad_federativa`: `CharField(max_length=100)`.
- `clave_delegacion_municipio`: `IntegerField`.
- `descripcion_delegacion_municipio`: `CharField(max_length=150)`.
- `clave_localidad`: `IntegerField` (Nullable).
- `descripcion_localidad`: `CharField(max_length=150)` (Nullable).
- `clave_colonia`: `IntegerField`.
- `descripcion_colonia`: `CharField(max_length=150)`.

3.1.3 Entidad **Producto** (Tabla **productos**)

Almacena la tipificación de productos financieros según CONDUSEF.

- `clave_producto`: `BigIntegerField` (Clave Primaria compuesta inyectada por catálogo oficial).
- `tipo_operacion`: `CharField(max_length=255)`.
- `producto`: `CharField(max_length=255)`.
- `subproducto`: `CharField(max_length=255)` (Nullable).

3.1.4 Entidad **Causa** (Tabla **causas**)

Tipificación regulatoria de la causa de la queja.

- `id`: `BigAutoField` (Clave Primaria).
- `clave_causa`: `IntegerField`.
- `descripcion_causa`: `TextField`.
- `producto`: `ForeignKey(Producto, on_delete=models.CASCADE, related_name='causas')` (Relación Muchos a Uno).

3.1.5 Entidad **Queja** (Tabla **quejas**)

Entidad transaccional núcleo del sistema.

- `folio`: `CharField(max_length=50)` (Clave Primaria física, generada de forma automatizada por el sistema).
- `fecha_consulta`: `DateField` (Fecha de registro o consulta de la reclamación).

- `estado: IntegerField` (Elecciones: 1 para 'Abierto/En proceso', 2 para 'Concluido/Cerrado').
 - `fecha_cierre: DateField` (Nullable, obligatoria únicamente si el estado de la queja se define como Concluido).
 - `producto: ForeignKey(Producto, on_delete=models.CASCADE, related_name='quejas')` (Relación Muchos a Uno).
 - `causa: ForeignKey(Causa, on_delete=models.CASCADE, related_name='quejas')` (Relación Muchos a Uno).
 - `medio: CharField(max_length=100, default='Teléfono')`.
 - `socio_referencia: ForeignKey(Socio, on_delete=models.CASCADE, null=True, blank=True, related_name='quejas')` (Relación Muchos a Uno, nullable).
-

3.2 Diseño de la Interfaz de Programación de Aplicaciones (API) y Controladores

La plataforma expone endpoints específicos para su consumo interno y procesamiento de datos.

3.2.1 Endpoint de Búsqueda Predictiva de Código Postal

- **Ruta:** `/socios/buscar-cp/`
- **Método HTTP:** `GET`
- **Parámetros de Entrada:**
 - `cp` (Query string, obligatorio, 5 dígitos numéricos).
- **Códigos de Estado y Respuestas JSON:**
 - **200 OK:** Petición exitosa. Retorna el estado, municipio y el catálogo de colonias asociadas.


```
{
  "estado": "Oaxaca",
  "municipio": "San Pedro Mixtepec Distrito 22",
  "colonias": [
    "Bajos de Chila",
    "Centro",
    "La Reforma"
  ]
}
```
 - **400 Bad Request:** Formato de código postal inválido. `{ "error": "Código postal inválido" }`

- **404 Not Found:** Código postal no existente en la base de datos nacional. { "error": "Código postal no encontrado" }

3.2.2 Controlador del Cierre Trimestral y Generador de Lotes

- **Ruta:** /cierre/
- **Método HTTP:** POST
- **Parámetros de Entrada:**
 - `queja_ids`: Arreglo de cadenas de texto inyectado en el cuerpo de la solicitud POST representando los folios a empaquetar.
 - `csrfmiddlewaretoken`: Token de seguridad inyectado en el cuerpo de la solicitud.
- **Códigos de Estado y Respuestas:**
 - **200 OK (Descarga de Archivo):** Retorna el cuerpo JSON adjunto con la cabecera `Content-Type: application/json` and `Content-Disposition: attachment; filename="lote_condusef.json"`.

```
[
  {
    "folio": "260601",
    "fecha_consulta": "2026-06-14",
    "estado": 2,
    "fecha_cierre": "2026-06-25",
    "producto": 1001,
    "causa": 1211,
    "medio": "Presencial"
  }
]
```

3.3 Lógica de Negocio y Algoritmos Críticos

3.3.1 Algoritmo de Generación Automática de Folio Consecutivo

El algoritmo se ejecuta a nivel del motor ORM de Django interceptando el flujo de guardado en base de datos. Asegura la secuencialidad y evita colisiones en entornos de operación diaria concurrente.

```
[Inicio de Guardado de Queja]
|
¿El objeto tiene Folio?
/      \
(Sí)    (No)
/      \
```

```

[Continuar con save]    Obtener fecha_consulta del objeto
                        |
                        Extraer YY = Dos dígitos del año (e.g.,
"26")
                        Extraer MM = Dos dígitos del mes (e.g.,
"06")
                        |
                        Consultar base de datos:
                        Contar quejas donde año y mes de consulta
coincidan
                        |
                        Calcular Consecutivo = Conteo + 1
                        |
                        Formatear NN = Consecutivo con relleno a 2
dígitos (e.g., "01")
                        |
                        Concatenar Folio = YY + MM + NN (e.g.,
"260601")
                        |
                        Asignar Folio al objeto
                        |
[Ejecutar super().save()]
                        |
                        [Fin]

```

3.3.2 Algoritmo de Validación de Consistencia de Estado de Queja

Regla crítica que garantiza que ninguna queja marcada como concluida sea registrada sin indicar cuándo finalizó el proceso de resolución.

1. **Entrada:** Formulario de queja enviado por el usuario.
2. **Validación en Backend ('Queja.clean()'):**
 - Si `estado` es numéricamente igual a 2 ('Concluido/Cerrado'):
 - Evaluar si el campo `fecha_cierre` tiene un valor nulo (`None`).
 - Si es nulo, arrojar un error de validación `ValidationError` asociándolo directamente al campo `fecha_cierre` en el formulario para indicar el error visual en el campo específico de la interfaz.
 - Si `estado` es numéricamente igual a 1 ('Abierto/En proceso'):
 - Permitir que `fecha_cierre` sea nula. En caso de contar con valor residual, limpiarlo y fijarlo como `None`.
3. **Salida:** Objeto validado listo para persistencia física.

4. DISEÑO DE LA INTERFAZ DE USUARIO (UI)

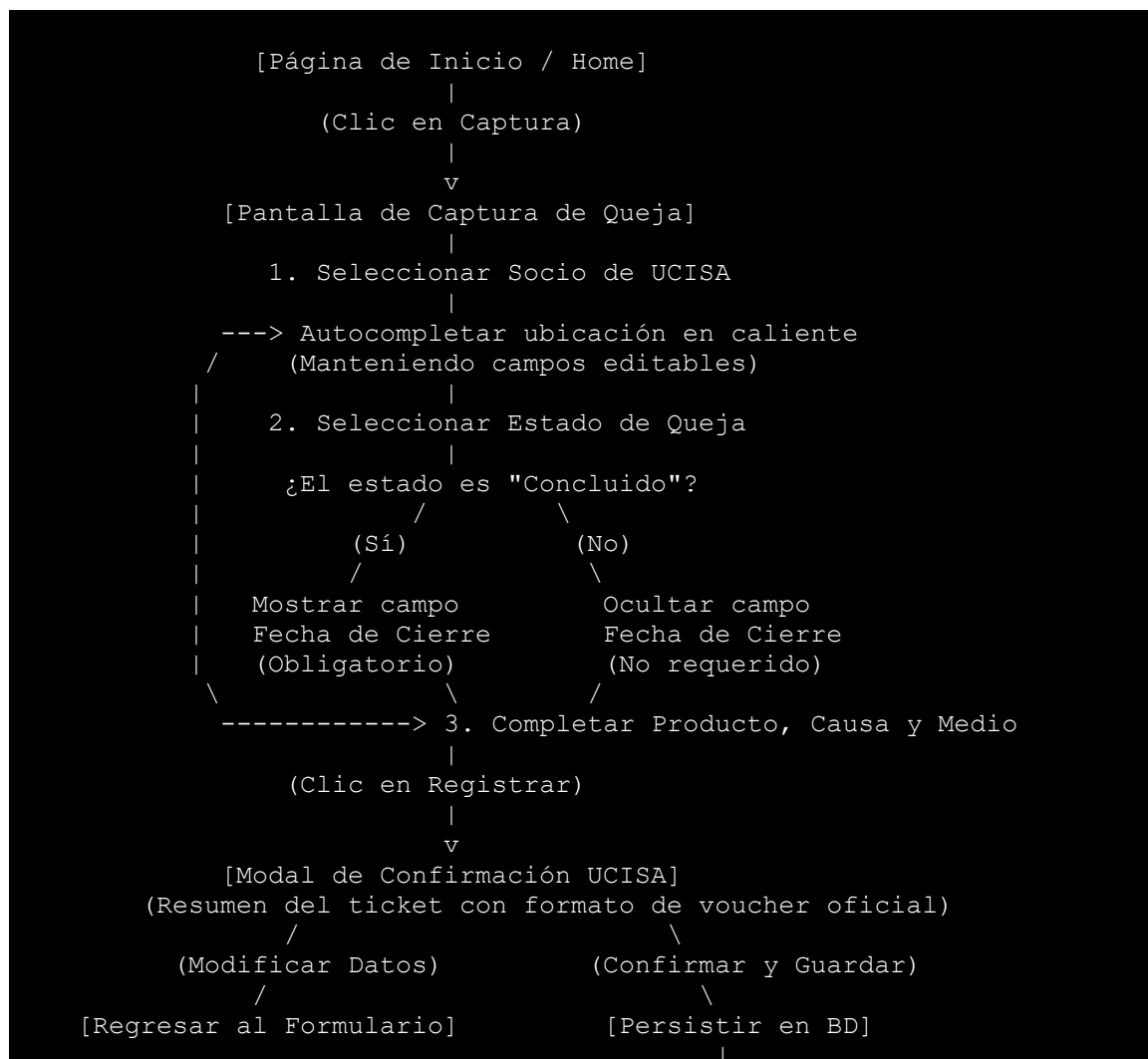
4.1 Filosofía de Diseño y Flujo de Navegación

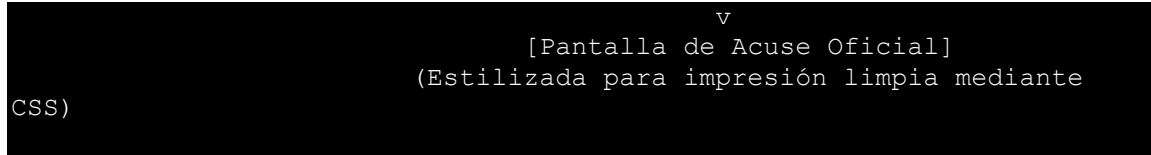
La interfaz se concibe bajo el concepto de **Diseño Corporativo Financiero**, priorizando la legibilidad, la sobriedad y la eficiencia en la captura masiva de datos en oficina.

4.1.1 Paleta de Colores y Tipografía:

- **Color Primario:** Azul Corporativo UCISA (#1c62c4). Utilizado en barras de navegación, botones principales y headers de paneles de control.
- **Color de Fondo Claro:** Gris claro (#f4f6f9), idóneo para mitigar la fatiga visual del operador durante jornadas prolongadas.
- **Modo Oscuro Integrado:** El sistema incluye un archivo CSS adaptativo que conmuta de forma limpia todas las variables de fondo de Bootstrap hacia negros y grises profundos al activar el botón selector. Este estado se almacena en el `localStorage` del cliente para mantener la preferencia del usuario en toda la sesión de trabajo.
- **Tipografía:** Fuente del sistema sans-serif optimizada para pantallas web corporativas: 'Segoe UI', Roboto, 'Helvetica Neue', Arial.

4.1.2 Flujo de Captura y Validación del Operador:





4.1.3 Hojas de Estilo para Impresión Física (Print Stylesheet):

El archivo de plantilla `ticket_queja.html` integra estilos CSS encapsulados dentro de una media query del tipo `@media print`. Esto asegura que al presionar el botón de impresión:

- La barra de navegación del sistema (`.ucisa-navbar`), los botones de acción del navegador ("Imprimir Acuse", "Volver al Inicio"), y los pies de página sean excluidos del plano de impresión (`display: none;`).
- Los bordes y sombras de la tarjeta principal se re-estilicen para eliminar efectos de degradado digital tridimensionales.
- Se reajusten los márgenes físicos de impresión para permitir que el acuse oficial se imprima directamente en papel bond tamaño carta o formato de miniprinter de ticket continuo, reduciendo el consumo de tinta de impresión corporativa.